

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: Dijkstra

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El programa presentado utiliza la biblioteca SFML (Simple and Fast Multimedia Library) para implementar una solución visual y funcional del algoritmo de Dijkstra. Este algoritmo es ampliamente conocido por calcular el camino más corto en un grafo ponderado, y aquí se aplica a una representación de laberinto en una cuadrícula. La visualización permite a los usuarios seleccionar puntos de inicio y fin en un laberinto interactivo y observar cómo el algoritmo determina el camino óptimo, resaltando este camino en pantalla.

La integración de SFML para renderizar el laberinto y el camino más corto hace que este programa sea no solo funcional, sino también visualmente intuitivo. Este diseño facilita tanto el aprendizaje del algoritmo de Dijkstra como su aplicación en problemas prácticos como navegación, planificación de rutas y juegos, al permitir una interacción directa con el entorno.

Desarrollo

Estructura del Programa

El programa se divide en tres componentes principales: el manejo del laberinto, la implementación del algoritmo de Dijkstra y la visualización interactiva. Cada una de estas partes juega un rol esencial en la funcionalidad y claridad del programa:

1. **Representación del Laberinto:** El laberinto está definido como una matriz bidimensional de enteros (`maze`), donde:
 - `0` representa espacios transitables.
 - `1` representa obstáculos. Esta representación simplifica la lógica del algoritmo y permite la fácil adaptación a diferentes configuraciones de laberintos.
2. **Algoritmo de Dijkstra:** El algoritmo implementado se encarga de calcular el camino más corto desde un punto de inicio hasta un punto de destino en el laberinto. Se utiliza una cola de prioridad (`std::priority_queue`) para explorar los nodos de menor costo acumulado primero. Las principales características del algoritmo incluyen:
 - Una matriz de costos mínimos (`dist`) para rastrear el costo más bajo a cada nodo.
 - Una matriz de nodos previos (`prev`) para reconstruir el camino una vez alcanzado el destino.

- Exploración de vecinos en las cuatro direcciones cardinales, verificando límites y transitabilidad.
- 3. **Interfaz Gráfica:** La biblioteca SFML maneja la visualización y la interacción del usuario. Algunas características clave son:
 - **Selección del inicio y fin:** Los usuarios pueden hacer clic en la cuadrícula para definir el punto de partida (verde) y el destino (rojo).
 - **Dibujado del laberinto:** Cada celda se representa como un rectángulo, con diferentes colores para obstáculos, camino y nodos especiales (inicio/fin).
 - **Camino más corto:** Una vez calculado, el camino se dibuja en celdas de color cian, resaltando el resultado del algoritmo.
 -

Implementación del Algoritmo

El algoritmo de Dijkstra está encapsulado en la función `dijkstra`, que toma como entrada las posiciones de inicio y fin. Este realiza lo siguiente:

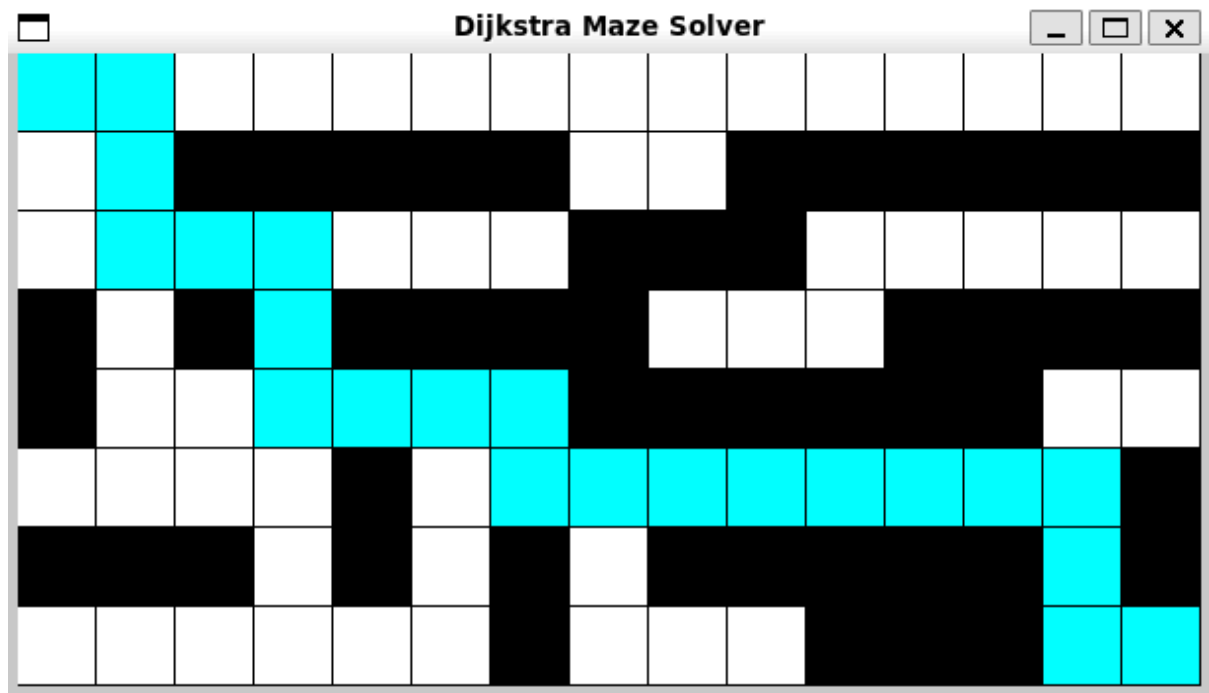
- Inicializa los costos de todas las celdas a infinito (`INT_MAX`) excepto el nodo de inicio, cuyo costo es 0.
- Utiliza una cola de prioridad para explorar los nodos en orden ascendente de costo acumulado.
- Actualiza los costos y caminos previos al encontrar rutas más cortas hacia nodos vecinos.
- Reconstruye el camino final desde el destino hacia el inicio utilizando la matriz `prev`.
-

Gestión de Gráficos

El uso de SFML permite representar de manera clara el estado del laberinto y el progreso del algoritmo:

- Las celdas se dibujan con un tamaño configurable (`CELL_SIZE`) y colores diferenciados para obstáculos, espacios libres, inicio, fin y el camino encontrado.
- El programa actualiza dinámicamente la ventana para reflejar la interacción del usuario y los resultados del algoritmo.
- Los clics del mouse permiten definir puntos de inicio y fin, lo que habilita la recalculación automática del camino.

Ejecución



Conclusión

Este programa ofrece una implementación funcional y visualmente atractiva del algoritmo de Dijkstra, utilizando la biblioteca SFML para crear una interfaz gráfica interactiva que facilita su comprensión y experimentación. La representación del laberinto como una cuadrícula simple y el uso de colores diferenciados para cada tipo de celda hacen que el proceso de búsqueda sea intuitivo y fácil de seguir. Además, la posibilidad de seleccionar dinámicamente los puntos de inicio y fin fomenta la exploración y prueba de diferentes escenarios.

El diseño modular y claro del código permite adaptaciones y mejoras, como la integración de diferentes algoritmos de búsqueda o la expansión a laberintos más complejos. En conjunto, el programa no solo es una herramienta útil para aprender sobre Dijkstra, sino que también tiene potencial para aplicaciones prácticas en áreas como videojuegos, navegación y resolución de problemas de rutas en grafos. Su combinación de eficiencia, interactividad y visualización lo convierte en un ejemplo destacado de cómo llevar conceptos algorítmicos a implementaciones prácticas y accesibles.

Código

```
#include <SFML/Graphics.hpp>
#include <queue>
#include <vector>
#include <iostream>
#include <utility>
#include <cmath>
#include <climits>
using namespace std;

// Laberinto definido como una matriz de enteros (0: espacio libre, 1: obstáculo)
vector<vector<int>> maze = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1},
    {0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0},
    {1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1},
    {1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0},
    {0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1},
    {1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1},
    {0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0}
};

const int CELL_SIZE = 40; // Tamaño de cada celda en píxeles

// Estructura para representar un nodo del laberinto
struct Node {
    int x, y, cost; // Coordenadas y costo acumulado para llegar a este nodo
    bool operator>(const Node &other) const { return cost > other.cost; } //
// Comparador para la cola de prioridad
};

// Implementación del algoritmo de Dijkstra para encontrar el camino más corto
vector<sf::Vector2i> dijkstra(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de costos mínimos desde el nodo inicial
    vector<vector<int>> dist(rows, vector<int>(cols, INT_MAX));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1, -1}));
    // Cola de prioridad para explorar los nodos
    priority_queue<Node, vector<Node>, greater<Node>> pq;

    // Inicializar el nodo de inicio
    pq.push({start.x, start.y, 0});
    dist[start.x][start.y] = 0;

    // Direcciones posibles para moverse (derecha, abajo, izquierda, arriba)
    vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Bucle principal del algoritmo
    while (!pq.empty()) {
        Node current = pq.top();
        pq.pop();

        // Si llegamos al nodo final, terminamos
        if (sf::Vector2i(current.x, current.y) == end)
            break;

        // Explorar los vecinos
        for (auto dir : directions) {
            int nx = current.x + dir.x; // Nueva posición en x
            int ny = current.y + dir.y; // Nueva posición en y

            // Verificar si el vecino está dentro de los límites y es transitable
```

```

        if (nx >= 0 && ny >= 0 && nx < rows && ny < cols && maze[nx][ny] == 0)
        {
            int new_cost = current.cost + 1; // Costo acumulado
            // Si encontramos un camino más corto, actualizamos
            if (new_cost < dist[nx][ny]) {
                dist[nx][ny] = new_cost;
                prev[nx][ny] = {current.x, current.y};
                pq.push({nx, ny, new_cost});
            }
        }
    }

    // Reconstrucción del camino desde el nodo final al inicio
    vector<sf::Vector2i> path;
    for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at = prev[at.x][at.y])
    {
        path.push_back(at);
    }
    reverse(path.begin(), path.end()); // Invertimos el camino para que vaya del
    inicio al final
    return path;
}

int main() {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Crear la ventana de renderizado
    sf::RenderWindow window(sf::VideoMode(cols * CELL_SIZE, rows * CELL_SIZE),
    "Dijkstra Maze Solver");

    sf::Vector2i start{-1, -1}, end{-1, -1}; // Coordenadas del inicio y fin
    bool selectingStart = true; // Bandera para determinar si estamos seleccionando
    el inicio

    // Bucle principal de la aplicación
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close(); // Cerrar ventana

            // Detectar clics del mouse para seleccionar inicio y fin
            if (event.type == sf::Event::MouseButtonPressed) {
                int x = event.mouseButton.y / CELL_SIZE; // Coordenada x en celdas
                int y = event.mouseButton.x / CELL_SIZE; // Coordenada y en celdas

                if (x >= 0 && y >= 0 && x < rows && y < cols && maze[x][y] == 0) {
                    if (selectingStart) {
                        start = {x, y}; // Asignar inicio
                        selectingStart = false; // Pasar a seleccionar el fin
                    } else {
                        end = {x, y}; // Asignar fin
                    }
                }
            }
        }

        window.clear(sf::Color::White); // Limpiar la ventana

        // Dibujar el laberinto
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE -
1));
                cell.setPosition(j * CELL_SIZE, i * CELL_SIZE);
            }
        }
    }
}

```

```

        // Color de las celdas
        if (maze[i][j] == 1)
            cell.setFillColor(sf::Color::Black); // Obstáculo
        else
            cell.setFillColor(sf::Color::White); // Espacio libre

        // Colorear inicio y fin
        if (start == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Green); // Inicio
        if (end == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Red); // Fin

        // Dibujar borde negro
        cell.setOutlineThickness(1);
        cell.setOutlineColor(sf::Color::Black);

        window.draw(cell); // Dibujar celda
    }

    // Dibujar el camino más corto si el inicio y fin están definidos
    if (start != sf::Vector2i{-1, -1} && end != sf::Vector2i{-1, -1}) {
        auto path = dijkstra(start, end);
        for (auto &pos : path) {
            sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE -
1));

            cell.setPosition(pos.y * CELL_SIZE, pos.x * CELL_SIZE);
            cell.setFillColor(sf::Color::Cyan); // Color del camino
            cell.setOutlineThickness(1);
            cell.setOutlineColor(sf::Color::Black);
            window.draw(cell); // Dibujar celda del camino
        }
    }

    window.display(); // Mostrar la ventana
}

return 0;
}

```